

Silent Serving-Layer Confounds in Measuring Tool-Call Protocol Fidelity of Small Local Coding Agents

Lijia Tang
Northeastern University
tang.lij@northeastern.edu

June 29, 2026

Abstract

Before a coding agent can decide which tool to use, it must emit a valid tool call: a parseable invocation of a tool that exists in the schema. We tried to measure how often small local models clear this protocol step, and found that the standard measurement is silently contaminated by the serving layer. With models served locally through Ollama, the default harness sends the OpenAI `tools=` request, which Ollama gates per model by a template flag unrelated to ability: some models have it accepted and return calls as text, some return native `tool_calls`, and others (Phi-3, Gemma-3) have the request rejected with HTTP 400. The harness logs a rejection as an ordinary non-call turn, so a naive analysis scores a capable model at 0% fidelity even though it never ran. Model non-responses are logged the same way, a second silent mechanism. On a second stack (llama.cpp) the same weights run on the same request, so the rejection is the stack’s policy, not the model. We then add a control arm that separates the serving channel from the prompt: for models the server accepts, injecting a plain-text tool list and call format into the prompt while keeping the native channel recovers most of the fidelity, so the default native path was under-measuring the model for lack of guidance, not for lack of ability; for Llama-3.2, which has real native support, the uniform text protocol instead lowers fidelity. The best configuration is therefore model-dependent, and any single fixed setup mismeasures some models. Grammar-constrained decoding removes the parse failures by construction but makes the weaker models loop without terminating, trading one failure mode for another. Finally, the fidelity number is not robust: turn-pooled and per-seed estimates differ by up to about 55 points because a single looping episode dominates the pool, on denominators as small as 8–16 turns over 8 seeds. We do not draw conclusions about scale, family, or reasoning; the data does not support them at this precision. The usable result is a checklist for measuring local agents: hold the serving interface fixed, separate a refused or empty request from a model non-call, and report per-seed rates with intervals. All experiments run on one laptop with no GPU.

1 Introduction

A coding-agent loop assumes the model can reliably emit a tool call the harness can execute. This is a protocol step, separate from the semantic step of choosing the right tool. We wanted to measure, on small local models, how often this step succeeds. The measurement turned out to be the problem.

With models served locally through Ollama’s OpenAI-compatible endpoint, the default harness sends the `tools=` parameter, and Ollama gates that request per model using a flag in the model template that does not reflect tool-use ability. On one fixed stack we see three behaviors. Qwen2.5-Coder (all sizes) accepts `tools=` and returns the call as text. Llama-3.2 returns native `tool_calls`.

Phi-3 and Gemma-3 have the request rejected with HTTP 400 “does not support tools”. The harness retries, gives up, and writes a generic error string into the trajectory as an assistant turn (§5), which a downstream per-turn analysis cannot distinguish from a model that chose not to call a tool. So a capable model is scored 0% because its request never ran, and the failure is silent.

This is not a claim that small models are good or bad at tool use. It is a claim that the standard local measurement does not measure the model. We make the point precise with a set of experiments and one negative result:

- **Native FC silently contaminates the measurement** through request rejection (Phi-3, Gemma-3 scored 0% without running) and through model non-responses, both logged as ordinary non-call turns.
- **A control arm separates the channel from the prompt.** For models the server accepts, keeping the native channel but adding a text tool list and call format to the prompt recovers most of the fidelity; for Llama-3.2, the uniform text protocol lowers it. The best configuration is model-dependent.
- **The fidelity number is not robust.** Turn-pooled and per-seed rates differ by up to about 55 points because one looping episode dominates the pool, on thin denominators.
- **A constrained-decoding baseline** makes every model emit a valid call by construction, but makes the weaker models loop without terminating, moving the failure rather than removing it.
- **Negative result.** We could not support claims about scale, family, or reasoning at this measurement precision, and we report why rather than asserting them.

The contribution is a measurement caveat and a checklist, not a leaderboard.

2 Related Work

Function-calling evaluation. BFCL [3] is the standard function-calling leaderboard and distinguishes native FC mode from prompting mode, where tools are supplied in the prompt; our text-tools condition is an instance of prompting mode, and our control arm sits between the two. BFCL tracks decode and empty-response states, but it evaluates through per-model handlers and does not address local serving stacks where the request is gated or rejected per model, which is the confound we isolate. It reports aggregate accuracy rather than a per-turn outcome composition on a held-fixed stack, and other multi-step and live tool-calling benchmarks share this hosted-model framing without targeting the local serving stack.

Constrained decoding as the local remedy. The standard fix for unparseable tool calls on local models is grammar-constrained or schema-guided decoding (e.g. GBNF grammars, Outlines, vLLM guided decoding, Ollama structured output). A small-model agent survey [4] argues these close much of the gap. We do not propose a fix; we include constrained decoding only as a baseline (§4) to show the protocol failures are removable by construction, and we measure how the unconstrained default setups mismeasure the model.

Tool hallucination and failure taxonomies. PA-Tool [2] names “schema misalignment”, the hallucination of absent tool names, which we observe as one outcome category. MAST [1] catalogs agent failures on capable models and does not address the local serving layer.

3 Method

Harness and per-turn taxonomy. We use an off-the-shelf ReAct coding-agent harness, an extension of LOCA-bench [5], on a fixed aggregation task that requires several tool calls. We label every assistant turn: **valid in-schema call**, **hallucinated call** (parseable, tool not found), **unparseable text**, **no-call prose**, and **non-response** (the harness wrote its retry-exhaustion error, §5). Protocol fidelity is the valid-in-schema-call rate over turns the model actually produced, so non-responses are excluded from the denominator and reported separately. We run 8 seeds per model and report both the turn-pooled rate and the per-seed mean (each episode weighted equally) with the standard error across seeds.

Three serving conditions. (i) *native*: harness default, sends `tools=`; on Ollama this is gated per model. (ii) *native+hint*: `tools=` still sent, plus a plain-text tool list and an explicit JSON call format with an allowed-name list injected into the prompt; this isolates the prompt guidance from the serving channel and is only defined for models the server accepts. (iii) *text-tools*: `tools=` dropped, the same text guidance in the prompt, calls parsed from text; uniform across all models.

Models. Qwen2.5-Coder at 0.5B, 1.5B, 3B, 7B, and 14B; Llama-3.2-3B, Phi-3-mini, Gemma-3-4B, Gemma-3-270m, served locally via Ollama, plus `deepseek-v4-flash` (cloud) as a high-capability anchor. Exact tags, quantization, and versions are in §7.

4 Results

Native FC mismeasures, two ways (Fig. 1, gray; Fig. 2). Under native FC, Phi-3 and Gemma-3 score 0%: 100% of their turns are rejected requests, so the model never ran (marked “rej” in Fig. 1). The second mechanism is non-response: Gemma-3-4B, for instance, emits one valid call per seed and then fails to respond, which the naive denominator would count against it (Fig. 2, gray band).

The rejection is stack-specific, not the model (Table 2). To check that the rejection is a property of the serving stack rather than the model or our configuration, we issue the same native `tools=` request to a second local server, `llama-server` from `llama.cpp` with `--jinja`, using the same GGUF weights pulled by Ollama. The two models Ollama rejects with HTTP 400, Phi-3 and Gemma-3, both return 200 and run on `llama.cpp`, so their native “0%” is specific to Ollama’s per-model gate and not the weights. For the models both stacks accept, the handling agrees: Qwen-0.5B returns a text-format call on both, and Llama-3.2 returns native `tool_calls` on both. The measured outcome for a fixed model and request therefore depends on the serving stack.

The prompt, not the channel, drives the accepted-model gap (Fig. 1). For every model the server accepts, adding the text hint while keeping the native channel raises per-seed fidelity substantially (Qwen 0.5B–14B: 0/38/23/59/60% native → 35/58/82/89/80% native+hint), and native+hint is close to the uniform text-tools rate. So the low native numbers reflect a default call path without explicit format guidance, not model inability. Llama-3.2 is the informative exception: native+hint reaches 82% but the uniform text-tools protocol drops it to 44%, because Llama-3.2 has real native `tool_calls` support that the text protocol discards. The best configuration is model-dependent, so no single fixed measurement setup is fair to all models.

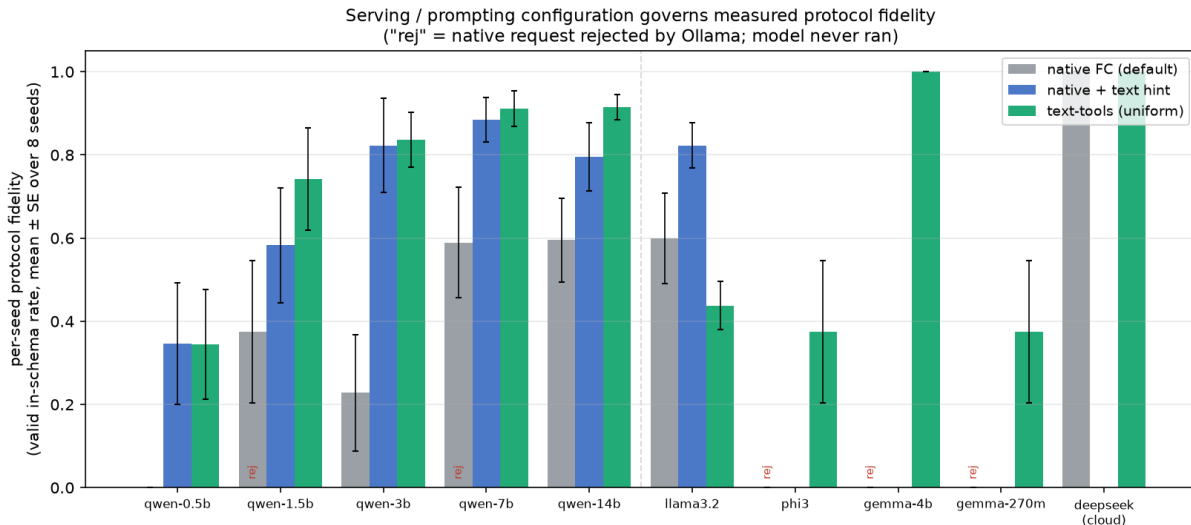


Figure 1: Per-seed protocol fidelity (valid in-schema rate, mean $\pm$ SE over 8 seeds) under three configurations. “rej” marks models whose native request was rejected by Ollama on every seed (the model never ran). For accepted models, adding a text hint to the native call (blue) recovers most of the fidelity, so the native channel itself is not the bottleneck; Llama-3.2 is the exception, where the uniform text-tools protocol (green) discards its real native support and lowers fidelity.

The fidelity number is not robust (Table 1). Turn-pooled and per-seed rates diverge sharply when a model produces one long looping episode. Qwen-0.5B under text-tools is 85% pooled but 34% per-seed: seven of eight episodes fail in one or two turns while a single 41-turn episode of repeated valid calls dominates the pool. Denominators are as small as 8–16 turns over 8 seeds for the weaker models, so per-model rates carry wide intervals (Fig. 1 error bars).

Constrained decoding removes the protocol failures, at a cost. As a baseline we constrain decoding to a JSON schema whose `name` field is an enum of the available tools (Ollama structured outputs). On a single tool-call step, all nine local models, including the three that scored lowest under text-tools (Llama-3.2, Phi-3, Gemma-3-270m), emit a valid in-schema call on 8 of 8 trials: parseability and in-schema names hold by construction. We do not run this as a full agentic condition because it removes the model’s ability to stop. Forced to emit a tool call on every turn, the weaker models never terminate, producing 600–900-turn loops within a single episode (Qwen-0.5B reached 869 assistant turns in one seed before we cut it off). Constrained decoding thus fixes the protocol layer we measure but trades an unparseable-call failure for a non-termination failure on weak models, which again locates the observed failure in the configuration rather than the model.

What we do not claim. The per-seed rates show some variation with model size within the Qwen family and across families, but the intervals overlap and the estimates depend on the pooling choice, so we do not claim a scale law, a family or tool-training effect, or a dissociation from reasoning. We ran a small no-tools reasoning probe while exploring those questions; it was not conclusive and we omit it to avoid over-reading.

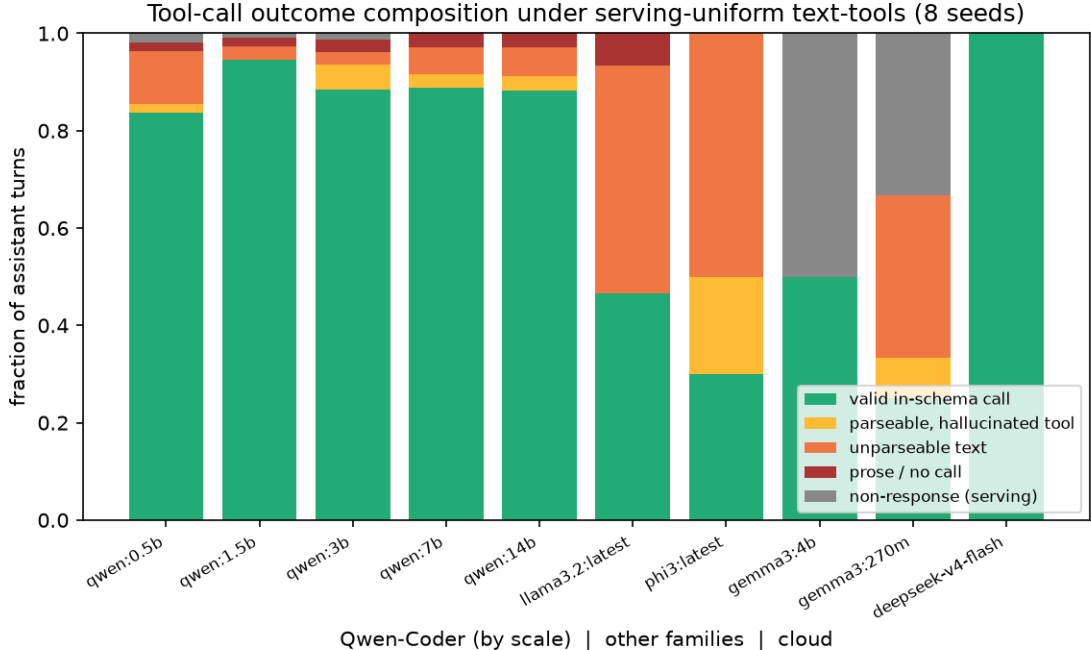


Figure 2: Per-turn outcome composition under the uniform text-tools protocol (8 seeds), including a non-response band. Gemma-3-4B’s apparent all-valid behavior is half non-responses; the fidelity rate (valid over produced turns) excludes these, which is why it must be reported alongside the non-response share, not alone.

5 The harness conflates serving failures with model non-calls

The contamination is silent because of how harnesses log failures. In the stack we used, when the request is rejected or retries are exhausted, the inference wrapper returns an assistant message whose content is a generic error string (e.g. "Failed to get response after multiple retries."); the error type is not persisted into the saved trajectory. A downstream per-turn analysis that reads the message stream therefore sees an ordinary non-call assistant turn, indistinguishable from a model that wrote prose, unless it string-matches the harness’s specific error text. We only separated rejection and non-response from genuine model behavior by adding such a match. Structured benchmarks like BFCL [3] do track decode and empty-response states, but agentic ReAct harnesses that persist only the message stream do not, and the local tool-gating that produces these rejections is specific to the serving stack rather than the benchmark. This is the concrete blind spot: the measurement instrument records a serving failure as a model failure.

6 Threats to Validity

Single task, and the cross-stack check is narrow. We use one aggregation task. We verified the rejection mechanism on a second stack (llama.cpp, Table 2), but the full per-seed measurements are Ollama-only; whether the prompt-vs-channel and robustness results carry over to llama.cpp or vLLM is untested. **Thin denominators and pooling.** The weaker models produce 8–16 turns over 8 seeds; per-seed and pooled rates diverge, and we report both rather than pick one. **Constrained-decoding baseline is single-turn.** Our constrained-decoding numbers come from

model	native FC seed (pool)	native+hint seed (pool)	text-tools seed (pool)	native rej	tt non- resp
Qwen2.5-Coder-0.5B	0 (0)	35 (29)	34 (85)	0	1
Qwen2.5-Coder-1.5B	38 (24)	58 (91)	74 (95)	0	1
Qwen2.5-Coder-3B	23 (79)	82 (96)	84 (90)	0	1
Qwen2.5-Coder-7B	59 (79)	89 (80)	91 (89)	0	0
Qwen2.5-Coder-14B	60 (74)	80 (71)	92 (88)	0	0
Llama-3.2-3B	60 (62)	82 (89)	44 (47)	0	0
Phi-3-mini	—	—	38 (30)	8	0
Gemma-3-4B	—	—	100 (100)	8	8
Gemma-3-270m	—	—	38 (38)	8	4
deepseek-v4-flash	100 (100)	—	100 (100)	0	0

Table 1: Protocol fidelity as per-seed mean with turn-pooled rate in parentheses, in percent, under three configurations. “native rej” counts seeds whose native request was rejected (-- means all 8, so native fidelity is undefined, not 0%). “tt non-resp” counts non-response turns under text-tools. The per-seed and pooled columns diverge most where a model loops (e.g. Qwen-0.5B text-tools 34 vs 85), which is why we report both.

model (same GGUF weights)	Ollama <code>tools=</code>	llama.cpp <code>tools=</code>
Phi-3-mini	HTTP 400, rejected	200, runs (text call)
Gemma-3-270m	HTTP 400, rejected	200, runs (text call)
Qwen2.5-Coder-0.5B	200, text-format call	200, text-format call
Llama-3.2-3B	200, native <code>tool_calls</code>	200, native <code>tool_calls</code>

Table 2: Same weights and the same native `tools=` request, two local serving stacks. The models Ollama rejects run on llama.cpp, so the rejection (and the 0% a naive analysis would record) is a property of the serving stack, not the model or a misconfiguration. Where both stacks accept the request, they agree.

a single tool-call step, not a full agentic run, because the agentic version did not terminate on weak models (§4); the single-turn result shows parseability is recoverable but does not measure multi-turn task behavior under constraint. **Hint design.** The text hint includes an allowed-name list; a weaker hint might recover less, so “the prompt drives it” is specific to this guidance. **Omitted probe.** We collected a reasoning probe but omit it because it was inconclusive; we therefore make no reasoning claim. **Few-seed fragility.** An earlier 3-seed pilot of ours showed a clean curve that 8 seeds dissolved; small-model rates are high-variance.

7 Reproducibility

All runs are local and free except the cloud anchor. Models are the Ollama tags `qwen2.5-coder` (sizes 0.5b through 14b), `llama3.2:latest`, `phi3:latest`, and `gemma3` (4b, 270m), at Ollama’s default quantization (Q4_K_M for these tags); the cloud anchor is `deepseek-v4-flash`, and the cross-stack check uses `llama-server` (llama.cpp) on the same GGUF blobs. The serving behavior is Ollama-version-dependent: the per-model `tools=` gating and the HTTP 400 “does not support tools” contract are properties of the Ollama release used, which must be pinned to reproduce the native-mode result. Decoding used temperature 1.0, top-*p* 1.0, and the harness retry and max-tool-

use limits quoted in the configuration. The three conditions correspond to the environment flags (`none`), `LOCA_TOOL_HINT=1`, and `LOCA_TEXT_TOOLS=1`; the task config, the per-turn classifier, the figures, and the aggregation task and probe specifications are released with the code.

8 Conclusion

Native function calling on a local server gates the tool-call request per model and records a refused or empty request as an ordinary non-call turn, so a capable model can be measured at 0% without ever running. A control arm shows that for accepted models the default native path under-measures fidelity for lack of prompt guidance rather than ability, and that forcing a uniform text protocol can instead hurt a model with real native support, so the fair configuration is model-dependent. Together with the large gap between pooled and per-seed estimates, the lesson is a measurement checklist for small or local agents: hold the serving interface fixed, separate a refused or empty request from a model non-call, and report per-seed rates with intervals. Otherwise the instrument measures the serving stack, not the model.

References

- [1] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.
- [2] Jonggeun Lee, Woojung Song, Jongwook Han, Haesung Pyun, and Yohan Jo. Don’t adapt small language models for tools; adapt tool schemas to the models. *arXiv preprint arXiv:2510.07248*, 2025. ACL 2026.
- [3] Shishir G. Patil, Huanzhi Mao, et al. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning (ICML)*, 2025. Leaderboard: <https://gorilla.cs.berkeley.edu/leaderboard.html>.
- [4] Raghav Sharma and Manan Mehta. Small language models for agentic systems: A survey of architectures, capabilities, and deployment trade-offs. *arXiv preprint arXiv:2510.03847*, 2025.
- [5] Weihao Zeng, Yuzhen Huang, and Junxian He. LOCA-bench: Benchmarking language agents under controllable and extreme context growth. *arXiv preprint arXiv:2602.07962*, 2026.